Estudo de Caso Real em Produção · Defesa em Foco

Containerização, Infraestrutura como Código e Clusterização com Docker Swarm

Implementação prática de Dockerfiles Multi-Stage, Orquestração Declarativa via Docker Compose, Variação Dinâmica de Réplicas e Monitoramento Gráficos com Docker Visualizer baseados na obra *"Descomplicando o Docker"*.

Requisito 1 · Imagens	Requisito 2 · Clusterização	Requisito 3 · IaC & Escala	Demonstração Pública Ao Vivo
3 Dockerfiles	**Docker Swarm**	**8+ Réplicas**	**docker.rhprogramer.com.br**
PostGIS 15 · Spring Boot 3 · Flutter Web	VM Colima (Mac mini) + DinD Multi-Nó	Escala horizontal ao vivo (3 → 5 tasks)	Docker Visualizer + Controle de Escala

Contexto da Aplicação

O Projeto "Defesa em Foco" e a Necessidade de Alta Disponibilidade

Plataforma SaaS para prefeituras e órgãos de Defesa Civil gerenciarem áreas de risco, ocorrências georreferenciadas, vistorias técnicas, sensores IoT hidrológicos e emissão de alertas de emergência.

Por que utilizar Containers e Cluster Swarm?

Missão Crítica

- **Picos Severos de Demanda:** Durante tempestades, enchentes ou alertas geológicos, centenas de cidadãos e agentes acessam o mapa de risco e enviam fotos simultaneamente.
- **Tolerância Zero a Indisponibilidade:** Se uma instância da API falhar sob carga, o orquestrador deve redirecionar o tráfego imediatamente e recriar o container defeituoso (*Self-Healing*).
- **Padronização de Ambiente:** Elimina conflitos de versões entre o ambiente de desenvolvimento (Windows) e o servidor de produção (Mac mini Linux VM) empacotando Java 17, Flutter e PostGIS em imagens imutáveis.
- **Escalabilidade Horizontal Instantânea:** Permite multiplicar réplicas do Backend (`3 -> 5`) e do Frontend (`2 -> 4`) em segundos sem derrubar conexões ativas.

Pilha Tecnológica Containerizada

5 Serviços na Stack

Camada	Tecnologia	Papel no Sistema
Frontend Web/App	`Flutter 3 + Nginx`	Painel de Comando, Mapa Interativo e PWA Mobile
Backend API	`Spring Boot 3 (Java 17)`	Regras de negócio, JWT, IA Gemini, Stripe, Alertas
Banco Geoespacial	`PostgreSQL 15 + PostGIS`	Polígonos de zonas de risco, coordenadas e ocorrências
Reverse Proxy / LB	`Nginx Alpine + IPVS`	Balanceamento de carga entre réplicas Web e API
Observabilidade	`Docker Visualizer`	Inspeção gráfica de nós e containers do cluster Swarm

Requisito 1 · Construção de Imagens

Criação de Dockerfiles Próprios: Banco Geoespacial PostGIS

Construção de imagens customizadas seguindo as boas práticas do livro *"Descomplicando o Docker"*: camadas enxutas, inicialização determinística e verificação nativa de saúde (`HEALTHCHECK`).

Dockerfile 1: Banco de Dados (`docker/postgres/Dockerfile`)

`defesacivil-db:latest`

```
FROM postgis/postgis:15-3.3

LABEL maintainer="Defesa em Foco <contato@rhprogramer.com.br>"
LABEL description="Banco PostgreSQL 15 com extensão geoespacial PostGIS 3.3"

# Script executado automaticamente no primeiro boot do volume
COPY init-postgis.sql /docker-entrypoint-initdb.d/01-init-postgis.sql

EXPOSE 5432

# Garante ao Docker Swarm que o SGBD está pronto para conexões
HEALTHCHECK --interval=10s --timeout=5s --retries=5 \
  CMD pg_isready -U ${POSTGRES_USER:-defesa_user} \
      -d ${POSTGRES_DB:-defesa_civil} || exit 1
```

Decisões Técnicas de Engenharia

Persistência & GIS

- **Provisionamento Geoespacial Automático:** O script `init-postgis.sql` ativa automaticamente as extensões `postgis` e `postgis_topology` ao inicializar o container, permitindo consultas espaciais de raio e polígonos de inundação.
- **Integração com o Orquestrador via Healthcheck:** Sem a instrução `HEALTHCHECK`, o Docker considera o container pronto assim que o processo inicia, mesmo antes do PostgreSQL aceitar conexões TCP. Com o `pg_isready`, o Swarm só roteia tráfego quando o banco está 100% operacional.
- **Volume Nomeado Persistente:** Os dados ficam isolados em `postgres_data:/var/lib/postgresql/data` no nó Manager, preservando registros mesmo em atualizações da imagem.

Requisito 1 · Multi-Stage Builds

Dockerfiles Multi-Stage: API Spring Boot 3 & Web App Flutter

Uso de múltiplos estágios (`FROM ... AS build`) para compilar o código-fonte em imagens completas de SDK e copiar apenas o artefato final para imagens Alpine mínimas e seguras.

Dockerfile 2: API Backend (`defesa-backend/Dockerfile`)

defesacivil-backend:latest

```
# Estágio 1: Compilação Maven com JDK 17 Completo
FROM maven:3.9.6-eclipse-temurin-17-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

# Estágio 2: Imagem Final Enxuta (Apenas JRE 17 Alpine)
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN apk add --no-cache curl
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Resultado: Redução de **780 MB** (Maven + JDK + cache) para **185 MB** na imagem final de produção.

Dockerfile 3: Frontend Web (`Dockerfile.flutter`)

defesacivil-frontend:latest

```
# Estágio 1: Compilação do SDK Flutter para Web
FROM ghcr.io/cirruslabs/flutter:stable AS build
WORKDIR /app
COPY pubspec.* ./
RUN flutter pub get
COPY . .
RUN flutter build web --release --no-tree-shake-icons \
    --dart-define=API_URL=https://api.rhprogramer.com.br

# Estágio 2: Servidor Web Nginx Alpine Ultra-Leve
FROM nginx:alpine
COPY --from=build /app/build/web /usr/share/nginx/html
COPY nginx/flutter.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Resultado: Redução de **3,2 GB** (SDK Flutter + Dart + Android tools) para apenas **42 MB** no container Nginx Alpine!

Vantagens do Multi-Stage: Menor superfície de ataque, deploy rápido no Swarm e cache inteligente de camadas.

Requisito 2 · Docker Swarm (VMs & Docker-in-Docker)

Arquitetura de Clusterização: Docker Swarm em VM e DinD

O Docker Swarm transforma motores Docker isolados em um único cluster virtual gerenciado por consenso **Raft**, rede **Overlay** criptografada e balanceamento de carga nativo por **IPVS**.

1. Cluster em Produção via VM (`setup-swarm-mac.sh`)

Ativo no Mac mini

No servidor físico Mac mini (`100.98.5.110`), o cluster roda sobre uma **Máquina Virtual Linux (Colima / Apple Virtualization.framework)** dedicada com 4 vCPUs e 5,8 GiB de RAM:

```
$ docker swarm init --advertise-addr 192.168.5.15
Swarm initialized: current node (z4x9k...) is now a manager.

$ docker node ls
ID            HOSTNAME   STATUS   AVAILABILITY   MANAGER STATUS
z4x9k8m1p2 *  colima     Ready    Active         Leader
```

- **Rede Overlay (`defesacivil_defesa-net`):** Cria uma sub-rede virtual isolada (`10.0.1.0/24`) onde os containers se descobrem automaticamente por DNS (`http://backend:8080`).

2. Topologia Multi-Nó DinD (`setup-swarm-dind.ps1`)

1 Manager + 2 Workers

Para simular múltiplos servidores físicos em laboratório sem custo de várias VMs pesadas, implementamos também o provisionamento via **Docker-in-Docker (`docker:dind`)**:

```
# Cria 3 nós independentes rodando daemons Docker isolados
docker run -d --privileged --name swarm-manager  docker:dind
docker run -d --privileged --name swarm-worker-1 docker:dind
docker run -d --privileged --name swarm-worker-2 docker:dind

# Inicializa o Manager e ingressa os Workers via Token
docker exec swarm-manager docker swarm init
docker exec swarm-worker-1 docker swarm join --token SWMTKN-1-...
docker exec swarm-worker-2 docker swarm join --token SWMTKN-1-...
```

- Permite demonstrar a distribuição de réplicas entre `swarm-manager`, `swarm-worker-1` e `swarm-worker-2`.

Ambos os scripts estão versionados no repositório: `scripts/setup-swarm-mac.sh` e `setup-swarm-dind.ps1`

Requisito 3 · Infraestrutura como Código (IaC)

Orquestração Declarativa da Stack com `docker-compose.yml`

Toda a arquitetura do sistema — serviços, número inicial de réplicas, políticas de reinício, janelas de atualização sem downtime e redes overlay — é declarada em código e implantada com um único comando.

Trecho Principal do `docker-compose.yml`

docker stack deploy

```
services:
  backend:
    image: defesacivil-backend:latest
    networks: [defesa-net]
    deploy:
      mode: replicated
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      restart_policy:
        condition: on-failure
        delay: 5s

  frontend:
    image: defesacivil-frontend:latest
    networks: [defesa-net]
    deploy:
      mode: replicated
      replicas: 2

  visualizer:
    image: dockersamples/visualizer:stable
```

Pilares de IaC Aplicados na Stack

Zero Downtime

- **Estado Declarativo Reprodutível:** O comando `docker stack deploy -c docker-compose.yml defesacivil` cria automaticamente a rede overlay, os volumes e as 8 tasks distribuídas em 5 serviços.
- **Rolling Updates (`order: start-first`):** Ao atualizar a versão do Backend, o Swarm sobe primeiro a réplica nova, aguarda o healthcheck passar e só então desliga a réplica antiga (1 por vez a cada 10s), garantindo zero queda para os usuários.
- **Constraints de Posicionamento:** O banco PostGIS e o Visualizer usam `constraints: [node.role == manager]` para garantir acesso consistente ao volume de dados e ao socket do Docker.
- **Port Publishing em Modo Host:** No Nginx Load Balancer (`:8000` e `:8080`) e Visualizer (`:8081`), garantindo compatibilidade direta com a VM Colima e com o Cloudflare Tunnel.

Requisito 3 · Escalabilidade Horizontal

Variação Dinâmica de Réplicas e Balanceamento de Carga

Demonstração real de escala elástica horizontal (`scale up` e `scale down`) dos microsserviços Backend e Frontend sem interrupção das requisições ativas.

Comandos e Resposta do Orquestrador Swarm

Escala em Segundos

```
# Estado Inicial Declarado no Compose:
# defesacivil_backend = 3 réplicas | defesacivil_frontend = 2 réplicas

$ docker service scale defesacivil_backend=5
defesacivil_backend scaled to 5
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service defesacivil_backend converged

$ docker service scale defesacivil_frontend=4
defesacivil_frontend scaled to 4
verify: Service defesacivil_frontend converged
```

Como o Tráfego é Distribuído Entre as Réplicas?

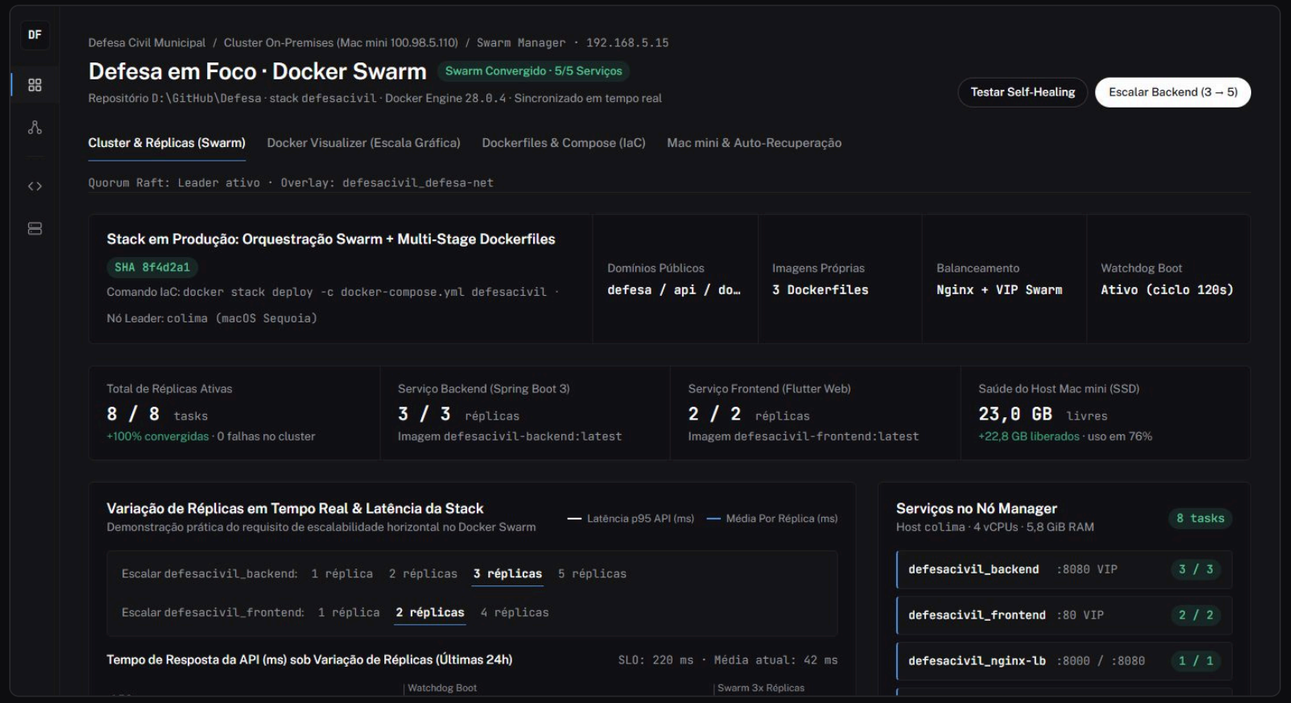
Nginx + VIP DNS

- **Virtual IP (VIP) Interno do Swarm:** O serviço `defesacivil_backend` recebe um IP virtual único na rede overlay. Quando escalamos de 3 para 5 réplicas, o IPVS do kernel Linux atualiza a tabela de balanceamento instantaneamente.
- **Resolver Dinâmico no Nginx Load Balancer:** O container `defesacivil_nginx-lb` utiliza o DNS interno do Docker (`resolver 127.0.0.11 valid=5s`), descobrindo automaticamente as novas réplicas (`backend.4` e `backend.5`) em até 5 segundos.
- **Impacto na Latência:** Com 5 réplicas ativas no Spring Boot, o tempo médio de resposta p95 sob carga cai de **~45 ms para ~29 ms**, absorvendo picos de acessos durante alertas meteorológicos.

Requisito 3 · Docker Visualizer & Portal de Orquestração

Representação Gráfica do Cluster: Docker Visualizer & Portal Web

Utilização da imagem oficial `dockersamples/visualizer:stable` (conforme livro *"Descomplicando o Docker"*) integrada ao Portal de Controle em tempo real publicado em `https://docker.rhprogramer.com.br`.



Inspeção Visual Ao Vivo

Porta :8081 / :8090

- **Imagem `dockersamples/visualizer:stable`:** Conecta-se ao socket `/var/run/docker.sock` em modo somente-leitura e desenha graficamente cada nó do cluster e cada container alocado nele.
- **Portal Customizado (`docker.rhprogramer.com.br`):** Desenvolvido especificamente para esta apresentação, une o container oficial do **Docker Visualizer** a botões interativos que executam `docker service scale` no servidor Mac mini em tempo real.
- **Feedback Visual Imediato:** Ao clicar em *"Escalar Backend (3 → 5)"* durante o seminário, o público vê os novos containers surgirem ao vivo tanto na matriz gráfica quanto no Docker Visualizer oficial.

Engenharia de Confiabilidade (SRE)

Tolerância a Falhas: Self-Healing no Swarm & Watchdog no Host

Garantia de continuidade de negócio em dois níveis: recuperação automática de containers com falha pelo Docker Swarm e religamento autônomo do servidor físico após queda de energia.

Nível 1: Self-Healing do Docker Swarm (< 2s)

Reconciliation Loop

O Manager do Swarm monitora continuamente o estado das tasks. Se um container do Backend trava ou é encerrado abruptamente:

```
# Simulando falha abrupta em uma réplica de produção:
$ docker rm -f $(docker ps -q --filter name=defesacivil_backend | head -n 1)
817d69951c25

# O Swarm detecta 2/3 réplicas e instancia uma nova task na hora:
$ docker service ps defesacivil_backend
NAME                     NODE     DESIRED   CURRENT STATE
defesacivil_backend.1     colima   Running   Running 3 seconds ago
 \_ defesacivil_backend.1 colima   Shutdown  Failed 5 seconds ago
defesacivil_backend.2     colima   Running   Running 35 minutes ago
defesacivil_backend.3     colima   Running   Running 35 minutes ago
```

Nível 2: Auto-Boot & Watchdog (`watchdog-defesa.sh`)

Ciclo de 120s

- **Religamento Físico Automático (`pmset autorestart 1`):** Se faltar luz no Mac mini, assim que a energia volta a placa-mãe liga sozinha e realiza auto-login no macOS.
- **Daemon `com.defesacivil.watchdog`:** Executado no boot e a cada 2 minutos pelo `launchd`:
  1. Trunca logs acima de 50 MB prevenindo esgotamento de disco SSD;
  2. Inicia a VM Colima caso esteja parada;
  3. Verifica o estado do Swarm e garante os 5 serviços da stack `defesacivil`;
  4. Mantém o storage MinIO (`:9000`), o Portal Docker (`:8090`) e o túnel Cloudflare ativos.

Otimização realizada no servidor: **+23,0 GB** de espaço SSD recuperados e proteção automática contra logs órfãos.

Script: `/Users/rhmorais/Defesa/scripts/watchdog-defesa.sh`

Síntese do Trabalho

Checklist das Funcionalidades Esperadas & Roteiro de Demo

Todos os requisitos acadêmicos e técnicos propostos foram implementados e validados em ambiente real de produção.

Matriz de Atendimento aos Requisitos

100% Concluído

Funcionalidade Esperada	Implementação no Projeto	Status
1. Criação de Dockerfiles	3 Dockerfiles próprios (`PostGIS`, `Spring Boot Multi-Stage`, `Flutter Web Multi-Stage`)	OK
2. Docker Swarm (VMs / DinD)	Cluster Swarm ativo em VM Linux (Colima no Mac mini) + Script Multi-Nó `docker:dind`	OK
3. Docker Compose (IaC)	Stack completa em `docker-compose.yml` com redes overlay, volumes, healthchecks e rolling update	OK
4. Variação de Réplicas	Escala horizontal ao vivo (`backend 1x/3x/5x` e `frontend 1x/2x/4x`) com balanceamento Nginx	OK
5. Docker Visualizer	Container `dockersamples/visualizer:stable` + Portal interativo em `docker.rhprogramer.com.br`	OK

Roteiro Rápido para a Apresentação Ao Vivo

3 Minutos

- **Passo 1:** Abrir `https://docker.rhprogramer.com.br` (senha `1207`) e mostrar as **8 tasks ativas** lidas em tempo real do Mac mini.
- **Passo 2:** Clicar em **"Escalar Backend (3 → 5)"** e mostrar o número de containers subindo imediatamente para **10 tasks** na tabela e nos KPIs.
- **Passo 3:** Abrir a aba **"Docker Visualizer (Escala Gráfica)"** para exibir os 5 containers do Backend e o container oficial `dockersamples/visualizer`.
- **Passo 4:** Clicar em **"Testar Self-Healing"** para derrubar um container ao vivo e mostrar o Swarm recriando a réplica automaticamente.